

# E commerce Microservices Software Design Document

Low Scale MVP Design and Eight Week Delivery Plan

| Document field     | Value                                   |
|--------------------|-----------------------------------------|
| Project            | E-commerce microservices                |
| Document version   | 1.0                                     |
| Status             | Ready for team and manager review       |
| Team               | Developer 1, Developer 2, Developer 3   |
| Service assignment | To be decided by the team before Week 2 |
| Planned duration   | 8 weeks                                 |
| Prepared date      | 2026-09-08                              |

Prepared for project manager review

## Document Summary

This Software Design Document defines the first release of the E-commerce microservices project. The first release is a backend-only system that three beginner developers can build in eight weeks. It covers user authentication, product and stock administration, order creation, order viewing, and customer cancellation.

The design deliberately keeps the first deployment small. The three services communicate through REST APIs, run with Docker Compose, and store data in separate PostgreSQL databases. The boundaries are suitable for later growth into event-driven communication, cloud deployment, caching, centralized observability, and additional services.

A manager can use this document to review scope, responsibilities, architecture, delivery dates, risks, and completion criteria. Developers can use it as the shared contract before the separate service implementation guides are prepared.

## Approval Record

| Role                     | Name | Decision                   | Date |
|--------------------------|------|----------------------------|------|
| Project manager          |      | Approve or request changes |      |
| Developer representative |      | Confirm technical scope    |      |

# Contents

The numbered headings below provide the shared design outline. Word navigation can also be used to jump between headings.

| Section | Subject                                 |
|---------|-----------------------------------------|
| 1       | Project Purpose and Scope               |
| 2       | Users Roles and Business Rules          |
| 3       | Functional Requirements                 |
| 4       | Quality Requirements                    |
| 5       | System Architecture                     |
| 6       | Service Interface Design                |
| 7       | Data Design                             |
| 8       | End to End Workflows                    |
| 9       | Security Design                         |
| 10      | Repository and Collaboration Design     |
| 11      | Development and Deployment Environments |
| 12      | Testing and Quality Assurance           |
| 13      | Logging Monitoring and Support          |
| 14      | Eight Week Delivery Plan                |
| 15      | Future Scale Roadmap                    |
| 16      | Risks and Mitigations                   |
| 17      | Acceptance and Handoff                  |

# 1 Project Purpose and Scope

## 1 1 Purpose

The team will build an e-commerce backend that demonstrates the complete path from user registration to order cancellation. The project teaches service boundaries, API design, authentication, database ownership, integration, automated testing, Docker, and collaborative GitHub development.

## 1 2 Goals

- Deliver a working backend MVP within eight weeks.
- Give each developer ownership of one independently runnable microservice.
- Keep API and database boundaries clear enough for future scaling.
- Demonstrate secure customer and administrator workflows.
- Use one GitHub repository with reviewed changes merged into a protected main branch.
- Provide enough automated tests and documentation for another beginner to run and understand the system.

## 1 3 Included in the MVP

| Area           | Included capability                                                                 |
|----------------|-------------------------------------------------------------------------------------|
| Users          | Register, log in, view and update profile, manage delivery addresses                |
| Security       | JWT authentication with CUSTOMER and ADMIN roles                                    |
| Catalog        | Browse active products and view product details                                     |
| Administration | Create, update, deactivate, and search products; adjust available stock             |
| Orders         | Create, list, view, and cancel eligible customer orders                             |
| Inventory      | Reserve stock during order creation and restore stock after cancellation            |
| Operations     | Docker Compose startup, health checks, logs, tests, and basic GitHub Actions checks |

## 1 4 Excluded from the MVP

- Payment processing, refunds, cards, wallets, taxes, invoices, and financial settlement.
- Shipment, warehouse, delivery tracking, returns, coupons, recommendations, reviews, and wish lists.
- Notification delivery by email, SMS, or push message.
- Kafka, Redis, API Gateway, service discovery, Kubernetes, and public cloud deployment.
- A browser or mobile frontend. Postman or another API client will be used for demonstrations.

Payment is deferred because a real payment service requires stronger security controls, idempotency, webhook processing, refund rules, and compliance work. The team should add it only after the order and inventory flows are reliable.

## 1 5 Assumptions and Constraints

| Item            | Decision                                                                       |
|-----------------|--------------------------------------------------------------------------------|
| Team experience | All developers are beginners and need explicit contracts and review checklists |
| Team size       | Three developers with one service owner per service after assignment           |
| Repository      | One GitHub monorepo shared by all collaborators                                |
| Runtime         | Java 21 and Spring Boot 3.x                                                    |
| Build           | Maven wrapper in every service                                                 |
| Database        | PostgreSQL with one separate database per service                              |
| Deployment      | Local Docker Compose for the MVP                                               |
| Traffic         | Learning and demonstration traffic, not production scale                       |

## 2 Users Roles and Business Rules

### 2 1 User Roles

| Role          | Allowed actions                                                                        | Restrictions                                                     |
|---------------|----------------------------------------------------------------------------------------|------------------------------------------------------------------|
| Guest         | Register, log in, browse active products                                               | Cannot create or view orders                                     |
| Customer      | Manage own profile and addresses; browse products; create, view, and cancel own orders | Cannot access another customer's data or administration APIs     |
| Administrator | Manage products and stock; browse catalog                                              | Does not automatically gain access to customer orders in the MVP |
| Service       | Order Service may call protected inventory reservation endpoints                       | Internal credentials must not be exposed to clients              |

### 2 2 Core Business Rules

| Rule   | Description                                                                               |
|--------|-------------------------------------------------------------------------------------------|
| BR 001 | Every email address is unique after lowercase normalization.                              |
| BR 002 | Passwords are never stored as plain text; Spring Security stores a one-way password hash. |
| BR 003 | Only an administrator can create, update, deactivate, or restock a product.               |

| Rule   | Description                                                                                                        |
|--------|--------------------------------------------------------------------------------------------------------------------|
| BR 004 | Customers can order only active products with sufficient available stock.                                          |
| BR 005 | The request to create an order contains product IDs and quantities, not trusted prices.                            |
| BR 006 | Product and Inventory Service supplies the current prices and changes stock in one database transaction.           |
| BR 007 | Order Service stores a price snapshot on each order item so later catalog price changes do not change past orders. |
| BR 008 | A customer can view or cancel only an order whose user ID matches the authenticated JWT subject.                   |
| BR 009 | Only a CONFIRMED order can be cancelled in the MVP.                                                                |
| BR 010 | Cancelling an order releases its reservation once and restores the exact quantities.                               |
| BR 011 | Repeated cancellation requests must return the existing CANCELLED state without adding stock twice.                |
| BR 012 | Services never read or write another service's database directly.                                                  |

## 2.3 Order Status Model

| Status    | Meaning                                                  | Allowed next status                    |
|-----------|----------------------------------------------------------|----------------------------------------|
| CREATED   | Order record is being prepared during the create request | CONFIRMED or removed if creation fails |
| CONFIRMED | Inventory is reserved and the order is accepted          | CANCELLED                              |
| CANCELLED | Customer cancelled and stock was restored                | No further transition                  |

## 3 Functional Requirements

| ID     | Capability   | Requirement                                                                 |
|--------|--------------|-----------------------------------------------------------------------------|
| FR 001 | Registration | A guest can register with name, unique email, and password.                 |
| FR 002 | Login        | A registered active user can receive a JWT after successful authentication. |
| FR 003 | Profile      | A customer can read and update the customer's own profile.                  |

| ID     | Capability        | Requirement                                                                            |
|--------|-------------------|----------------------------------------------------------------------------------------|
| FR 004 | Addresses         | A customer can create, read, update, and delete the customer's own delivery addresses. |
| FR 005 | Browse products   | A guest or authenticated user can list and search active products with pagination.     |
| FR 006 | Product details   | A guest or authenticated user can view an active product by ID.                        |
| FR 007 | Admin products    | An administrator can create, edit, deactivate, and view all products.                  |
| FR 008 | Admin inventory   | An administrator can increase or decrease available stock with validation.             |
| FR 009 | Reserve inventory | Order Service can atomically reserve multiple product quantities.                      |
| FR 010 | Release inventory | Order Service can release a reservation once after cancellation or compensation.       |
| FR 011 | Create order      | A customer can submit product IDs, quantities, and a delivery address snapshot.        |
| FR 012 | List orders       | A customer can list the customer's own orders with pagination.                         |
| FR 013 | Order details     | A customer can view one owned order with items, totals, address, and status.           |
| FR 014 | Cancel order      | A customer can cancel an owned CONFIRMED order and receive the updated order.          |
| FR 015 | Health status     | Every service exposes a health endpoint for local startup checks.                      |
| FR 016 | API documentation | Every service exposes an OpenAPI description and Swagger UI in the local environment.  |

## 4 Quality Requirements

The MVP is a learning system, so the quality targets below are acceptance targets for a local demonstration. They are not production service-level agreements.

| ID      | Area              | Acceptance target                                                                                          |
|---------|-------------------|------------------------------------------------------------------------------------------------------------|
| NFR 001 | Understandability | A new graduate can clone the repository, follow the README, and run the system without undocumented steps. |

| ID      | Area            | Acceptance target                                                                                       |
|---------|-----------------|---------------------------------------------------------------------------------------------------------|
| NFR 002 | Security        | Protected APIs reject missing, invalid, or expired JWTs. Role checks protect administrator operations.  |
| NFR 003 | Privacy         | Logs do not contain passwords, password hashes, full JWTs, or database credentials.                     |
| NFR 004 | Reliability     | Inventory reserve and release operations are transactional and safe against duplicate requests.         |
| NFR 005 | Performance     | During the demo, common API requests should normally complete within two seconds on a developer laptop. |
| NFR 006 | Maintainability | Controllers, services, repositories, domain objects, and API models have separate responsibilities.     |
| NFR 007 | Testability     | Business rules have unit tests and important APIs have integration tests.                               |
| NFR 008 | Portability     | The complete local environment starts through Docker Compose with documented prerequisites.             |
| NFR 009 | Observability   | Every request has clear logs and a correlation ID that is passed between Order and Product services.    |
| NFR 010 | Compatibility   | API changes preserve the /api/v1 prefix or are reviewed as breaking changes.                            |

## 5 System Architecture

### 5 1 Architecture Decision

The MVP uses three independently runnable Spring Boot applications in one repository. Each application owns its database and exposes a versioned REST API. Order Service calls Product and Inventory Service for stock reservation and release. The client calls each service directly during local development because an API Gateway is outside the first-release scope.

Low scale MVP architecture

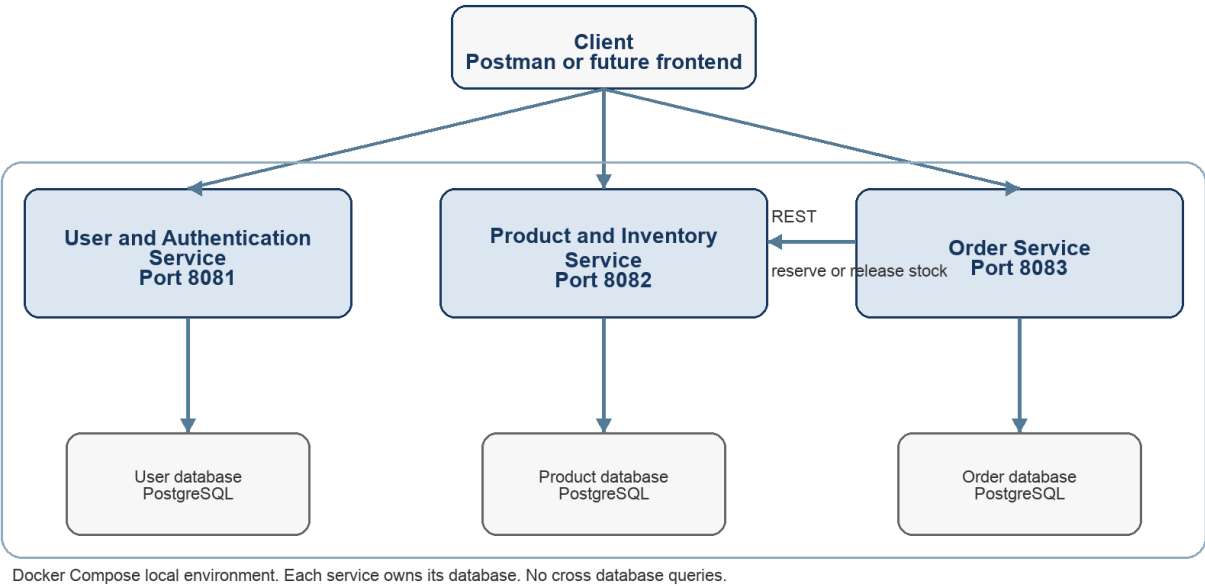


Figure 1 Low scale MVP architecture

5 2 Service Responsibilities

| Service                         | Owns                                                                          | Does not own                                      |
|---------------------------------|-------------------------------------------------------------------------------|---------------------------------------------------|
| User and Authentication Service | Credentials, user profile, addresses, roles, JWT creation                     | Products, stock, orders                           |
| Product and Inventory Service   | Categories, products, current prices, stock, inventory reservations           | Users, passwords, customer orders                 |
| Order Service                   | Orders, order items, price snapshots, address snapshots, order status history | Credentials, catalog master data, available stock |

5 3 Communication Rules

- Client-to-service and service-to-service communication uses JSON over HTTP.
- Order Service uses a configurable Product Service base URL. No URL is hard-coded in application code.
- Service calls have connection and response timeouts. They do not retry non-idempotent requests automatically.
- The X-Correlation-ID header is accepted from the client or created at entry and forwarded to downstream calls.
- Internal inventory endpoints require a service credential in addition to normal network isolation in Docker Compose.
- A service failure returns a controlled error response; stack traces are logged internally and are not sent to clients.

## 5 4 Data Ownership and Consistency

No database transaction can span all three services. Product and Inventory Service performs stock changes inside its own PostgreSQL transaction. Order Service compensates for a failure by calling the release endpoint when stock was reserved but the order could not be saved. Reservation and cancellation operations carry stable identifiers so repeated requests do not change stock more than once.

## 5 5 Technology Stack

| Layer         | MVP choice                                                       | Reason                                                             |
|---------------|------------------------------------------------------------------|--------------------------------------------------------------------|
| Language      | Java 21                                                          | Current long-term-support Java baseline for the project            |
| Framework     | Spring Boot 3.x                                                  | Web, validation, security, data access, and health support         |
| Build         | Maven wrapper                                                    | Same build command on every machine and in CI                      |
| Database      | PostgreSQL                                                       | Relational transactions fit users, products, inventory, and orders |
| Security      | Spring Security and JWT                                          | Stateless authentication and role checks                           |
| API           | REST and JSON with OpenAPI                                       | Simple to test and document for beginners                          |
| Local runtime | Docker and Docker Compose                                        | Repeatable multi-service startup                                   |
| Tests         | JUnit, Mockito, Spring Boot Test, Testcontainers where practical | Unit and integration coverage                                      |
| CI            | GitHub Actions                                                   | Run builds and tests on pull requests                              |

## 6 Service Interface Design

The endpoint catalog below defines the shared interface. Detailed request models, validation annotations, classes, and test cases will appear in the separate service implementation guides.

### 6 1 User and Authentication Service

| Method | Path                  | Access            | Purpose                          |
|--------|-----------------------|-------------------|----------------------------------|
| POST   | /api/v1/auth/register | Public            | Create a CUSTOMER account        |
| POST   | /api/v1/auth/login    | Public            | Authenticate and return a JWT    |
| GET    | /api/v1/users/me      | CUSTOMER or ADMIN | Return the authenticated profile |

| Method | Path                            | Access            | Purpose                                |
|--------|---------------------------------|-------------------|----------------------------------------|
| PUT    | /api/v1/users/me                | CUSTOMER or ADMIN | Update name and allowed profile fields |
| GET    | /api/v1/users/me/addresses      | CUSTOMER          | List owned addresses                   |
| POST   | /api/v1/users/me/addresses      | CUSTOMER          | Create an address                      |
| PUT    | /api/v1/users/me/addresses/{id} | CUSTOMER          | Update an owned address                |
| DELETE | /api/v1/users/me/addresses/{id} | CUSTOMER          | Delete an owned address                |

## 6 2 Product and Inventory Service

| Method | Path                                             | Access        | Purpose                                             |
|--------|--------------------------------------------------|---------------|-----------------------------------------------------|
| GET    | /api/v1/products                                 | Public        | Search and page through active products             |
| GET    | /api/v1/products/{id}                            | Public        | View one active product                             |
| GET    | /api/v1/admin/products                           | ADMIN         | Search all products including inactive records      |
| POST   | /api/v1/admin/products                           | ADMIN         | Create a product                                    |
| PUT    | /api/v1/admin/products/{id}                      | ADMIN         | Update product details or current price             |
| PATCH  | /api/v1/admin/products/{id}/stock                | ADMIN         | Adjust available stock by a signed quantity         |
| DELETE | /api/v1/admin/products/{id}                      | ADMIN         | Deactivate a product without deleting history       |
| POST   | /internal/v1/inventory/reservations              | ORDER SERVICE | Atomically reserve stock and return price snapshots |
| POST   | /internal/v1/inventory/reservations/{id}/release | ORDER SERVICE | Release a reservation once                          |

## 6 3 Order Service

| Method | Path                       | Access   | Purpose                                  |
|--------|----------------------------|----------|------------------------------------------|
| POST   | /api/v1/orders             | CUSTOMER | Create and confirm an order              |
| GET    | /api/v1/orders             | CUSTOMER | List the authenticated customer's orders |
| GET    | /api/v1/orders/{id}        | CUSTOMER | View an owned order                      |
| POST   | /api/v1/orders/{id}/cancel | CUSTOMER | Cancel an eligible owned order           |

## 6 4 Common API Rules

| Rule       | Standard                                                                           |
|------------|------------------------------------------------------------------------------------|
| Media type | application/json                                                                   |
| Versioning | Public APIs begin with /api/v1 and internal APIs with /internal/v1                 |
| Identity   | Authorization: Bearer <JWT> for protected APIs                                     |
| Time       | UTC ISO 8601 values such as 2026-09-08T14:30:00Z                                   |
| Money      | Decimal values with currency code; never use floating-point types for calculations |
| Pagination | page is zero-based; size has a documented maximum; response includes totals        |
| Tracing    | X-Correlation-ID is returned in every response                                     |
| Validation | Invalid input returns HTTP 400 with field-level details                            |
| Ownership  | An inaccessible resource may return 404 to avoid revealing another user's data     |

## 6 5 Error Response

All services use the same error shape so clients do not need service-specific error parsing.

| Field         | Example                           | Meaning                              |
|---------------|-----------------------------------|--------------------------------------|
| timestamp     | 2026-09-08T14:30:00Z              | UTC time of the failure              |
| status        | 409                               | HTTP status number                   |
| code          | INSUFFICIENT_STOCK                | Stable application error code        |
| message       | Requested quantity is unavailable | Safe explanation for the client      |
| path          | /api/v1/orders                    | Request path                         |
| correlationId | b6bd...                           | Identifier used to find related logs |
| fieldErrors   | quantity: must be at least 1      | Optional validation details          |

## 7 Data Design

Every table uses a generated UUID primary key unless stated otherwise. Database migrations are stored with each service and run through Flyway. Timestamps use UTC. Foreign keys exist only inside one service database. IDs received from another service are stored as values without cross-database foreign keys.

## 7 1 User Database

| Table     | Important fields                                                          | Notes                                               |
|-----------|---------------------------------------------------------------------------|-----------------------------------------------------|
| users     | id, email, password_hash, full_name, role, active, created_at, updated_at | Unique index on normalized email                    |
| addresses | id, user_id, line1, line2, city, state, postal_code, country, is_default  | Owned by one user; validate required address fields |

## 7 2 Product Database

| Table                  | Important fields                                                                         | Notes                                                     |
|------------------------|------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| categories             | id, name, active, created_at                                                             | Category name is unique                                   |
| products               | id, category_id, sku, name, description, price, currency, active, created_at, updated_at | Unique SKU; price is decimal                              |
| inventory              | product_id, available_quantity, version, updated_at                                      | Nonnegative quantity; version protects concurrent updates |
| inventory_reservations | id, order_reference, status, created_at, released_at                                     | Unique order reference and idempotent status              |
| reservation_items      | reservation_id, product_id, quantity, unit_price                                         | Stores reserved quantity and current price snapshot       |
| stock_adjustments      | id, product_id, quantity_change, reason, changed_by, created_at                          | Audit trail for administrator changes                     |

## 7 3 Order Database

| Table                | Important fields                                                                                    | Notes                                               |
|----------------------|-----------------------------------------------------------------------------------------------------|-----------------------------------------------------|
| orders               | id, user_id, reservation_id, status, total_amount, currency, address fields, created_at, updated_at | Index user_id and created_at; address is a snapshot |
| order_items          | id, order_id, product_id, product_name, sku, quantity, unit_price, line_total                       | Product details and price are snapshots             |
| order_status_history | id, order_id, old_status, new_status, reason, changed_at                                            | Supports audit and debugging                        |

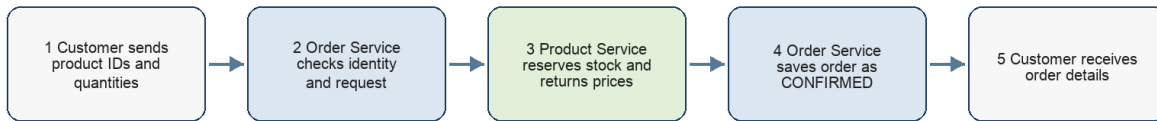
## 7 4 Data Retention and Deletion

The MVP deactivates products instead of deleting them because existing orders may contain those product IDs. User and order deletion workflows are outside scope. Development data may be reset through documented local scripts, but the team must not commit real personal data or database files to GitHub.

## 8 End to End Workflows

### Order creation and cancellation flows

#### Create order



#### Cancel order



Figure 2 Order creation and cancellation flows

### 8 1 Registration and Login

1. The client sends registration details to User and Authentication Service.
2. The service normalizes the email, validates the request, hashes the password, and stores a CUSTOMER account.
3. The customer sends email and password to the login endpoint.
4. The service verifies the password and returns a signed JWT containing the user ID, role, issue time, and expiry time.
5. The client sends the token in the Authorization header for protected operations.

### 8 2 Product Administration

1. An administrator logs in and receives an ADMIN JWT.
2. The administrator creates a product with a category, SKU, price, currency, and starting stock.
3. Product and Inventory Service validates the role and stores product and inventory records in one local transaction.
4. The public catalog shows only active products.
5. Each stock adjustment creates an audit record with quantity change, reason, and administrator identity.

### 8 3 Create Order

1. The customer sends product IDs, quantities, and a selected delivery address to Order Service.
2. Order Service reads the customer ID from the JWT and validates quantities and address data.

3. Order Service creates an order reference and requests one inventory reservation for all requested items.
4. Product and Inventory Service locks or version-checks the inventory rows, verifies every product, calculates price snapshots, decreases stock, and stores the reservation in one transaction.
5. Order Service saves the order, item snapshots, total, reservation ID, and CONFIRMED status.
6. If Order Service cannot save after reservation, it calls the release endpoint as compensation and logs the correlation ID.
7. The confirmed order response is returned to the customer.

## 8 4 Cancel Order

1. The customer sends a cancellation request for an order ID.
2. Order Service checks that the order belongs to the authenticated customer and is CONFIRMED.
3. Order Service asks Product and Inventory Service to release the stored reservation ID.
4. Product and Inventory Service restores each reserved quantity and marks the reservation RELEASED in one transaction.
5. Order Service changes the order to CANCELLED and adds a status-history record.
6. A repeated cancellation returns the existing cancelled order and does not release stock again.

## 8 5 Failure Handling

| Failure                            | Expected behavior                                           |
|------------------------------------|-------------------------------------------------------------|
| Invalid or expired JWT             | Return 401; make no data change                             |
| Wrong role                         | Return 403; make no data change                             |
| Product missing or inactive        | Return 409 from order creation with a stable error code     |
| Insufficient stock                 | Reserve nothing and return 409                              |
| Product Service unavailable        | Return 503; Order Service does not create a confirmed order |
| Order save fails after reservation | Call release as compensation and log an error for follow-up |
| Release temporarily fails          | Keep order CONFIRMED, return 503, and allow a safe retry    |
| Duplicate reserve or release       | Return the original result without changing stock twice     |

# 9 Security Design

## 9 1 Authentication

User and Authentication Service verifies credentials and signs JWT access tokens. For the MVP, services use an HS256 shared secret supplied through environment variables. The secret is never committed to GitHub. A later release should move to asymmetric signing and a public key or JWKS endpoint so verification services do not hold the signing key.

## 9 2 Authorization

| Control              | Implementation                                                                       |
|----------------------|--------------------------------------------------------------------------------------|
| Endpoint roles       | Spring Security rules protect CUSTOMER, ADMIN, and internal service endpoints        |
| Resource ownership   | Order and address queries include the authenticated user ID                          |
| Administrator access | Only ADMIN tokens can change products or stock                                       |
| Internal calls       | Order Service uses a separate service credential for inventory reservation endpoints |
| Default policy       | Any endpoint not explicitly public is authenticated                                  |

## 9 3 Credential and Data Protection

- Hash passwords with Spring Security's recommended password encoder configuration.
- Keep database passwords, JWT secrets, and service credentials in untracked .env files for local work.
- Commit a .env.example file containing names and safe placeholders only.
- Use HTTPS when the system leaves a developer laptop; HTTP is accepted only inside the local MVP environment.
- Validate and limit request sizes, string lengths, quantities, page sizes, and identifiers.
- Do not log credentials, JWTs, password fields, or full private address payloads.
- Return safe messages to clients and keep exception details in server logs.

## 9 4 Known MVP Security Limitations

The MVP does not include refresh tokens, account lockout, email verification, password reset, key rotation, centralized secrets, rate limiting, or external identity providers. These controls are required before any public production deployment.

# 10 Repository and Collaboration Design

## 10 1 Monorepo Structure

The project uses one GitHub repository. Each service remains independently buildable and owns its own source code, migrations, tests, and README.

```
ecommerce-microservices/  
  user-service/  
  product-inventory-service/  
  order-service/  
  docs/  
  postman/  
  .github/workflows/  
  docker-compose.yml  
  .env.example  
  README.md
```

## 10 2 Branch and Pull Request Rules

| Rule            | Team practice                                                                             |
|-----------------|-------------------------------------------------------------------------------------------|
| Main branch     | Contains stable integrated code. Direct pushes are disabled through branch protection.    |
| Feature branch  | Use feature/<service>/<short-description>, for example feature/order/create-order.        |
| Bug fix branch  | Use fix/<service>/<short-description>.                                                    |
| Pull request    | Describe the change, test evidence, API or database impact, and screenshots where useful. |
| Review          | At least one teammate reviews and approves before merge.                                  |
| Checks          | All affected Maven tests and the GitHub Actions workflow must pass.                       |
| Merge           | Use squash merge so one pull request becomes one clear commit on main.                    |
| Synchronization | Update the feature branch from main before final approval if conflicts exist.             |

## 10 3 Service Ownership

Developer 1, Developer 2, and Developer 3 will choose service ownership before Week 2. Ownership means leading design and implementation for that service. It does not permit merging without review, and every developer must understand the shared API contracts.

| Service                         | Primary owner                       | Required reviewers           |
|---------------------------------|-------------------------------------|------------------------------|
| User and Authentication Service | To assign from Developer 1, 2, or 3 | At least one other developer |
| Product and Inventory Service   | To assign from Developer 1, 2, or 3 | At least one other developer |
| Order Service                   | To assign from Developer 1, 2, or 3 | At least one other developer |
| Docker Compose and shared docs  | Shared                              | All developers               |

## 10 4 Change Control

- Record work as GitHub issues with an owner and acceptance criteria.
- Agree on request and response changes before one service depends on them.
- Treat database migration files as immutable after they reach main; add a new migration for later changes.
- Review changes to shared configuration and docker-compose.yml with all affected service owners.
- Tag the Week 8 demonstration build as v1.0.0 after acceptance checks pass.

# 11 Development and Deployment Environments

## 11 1 Required Tools

| Tool                                        | Purpose                                             |
|---------------------------------------------|-----------------------------------------------------|
| Git and GitHub account                      | Clone, branch, review, and merge code               |
| Java 21 JDK                                 | Compile and run Spring Boot services outside Docker |
| Docker Desktop or compatible Docker runtime | Run services and databases together                 |
| IDE                                         | IntelliJ IDEA Community, VS Code, or Eclipse        |
| Postman or similar client                   | Run end-to-end API demonstrations                   |

## 11 2 Local Ports

| Component                     | Host port | Container port |
|-------------------------------|-----------|----------------|
| User Service                  | 8081      | 8081           |
| Product and Inventory Service | 8082      | 8082           |
| Order Service                 | 8083      | 8083           |
| User PostgreSQL               | 5433      | 5432           |
| Product PostgreSQL            | 5434      | 5432           |
| Order PostgreSQL              | 5435      | 5432           |

## 11 3 Configuration

Each service reads configuration from `application.yml` with environment-variable overrides. Profiles may include local and test. Database URLs, passwords, JWT secrets, Product Service URL, timeouts, and internal service credentials must be configurable.

## 11 4 Startup and Health

- `docker compose up --build` starts the databases and all services.
- Database health checks prevent an application from starting before its database is ready.
- Spring Boot Actuator exposes a health endpoint for each service.
- The root README lists prerequisites, startup, sample users, API documentation URLs, test commands, and shutdown steps.
- The Postman collection runs registration, login, product setup, order creation, order viewing, cancellation, and stock verification in order.

## 12 Testing and Quality Assurance

### 12 1 Test Levels

| Test level                | Purpose                                               | Minimum examples                                                               |
|---------------------------|-------------------------------------------------------|--------------------------------------------------------------------------------|
| Unit tests                | Verify business logic without network or database     | Password rules, totals, status transitions, stock validation, ownership checks |
| Repository tests          | Verify mappings, queries, constraints, and migrations | Unique email, product search, order query by user, inventory version update    |
| API integration tests     | Verify validation, security, JSON, and status codes   | Register, login, admin denial, insufficient stock, unauthorized order access   |
| Service integration tests | Verify Order to Product communication                 | Reserve success, reserve rejection, release, downstream timeout                |
| End-to-end test           | Verify the complete Docker Compose system             | Admin adds stock; customer orders; cancellation restores stock                 |

### 12 2 Pull Request Quality Gate

- The affected service compiles with its Maven wrapper.
- Unit and integration tests pass.
- New or changed behavior has tests.
- OpenAPI documentation and the service README match the implementation.
- No secret, generated database file, IDE metadata, or build output is committed.
- A teammate confirms naming, validation, error behavior, and database migration safety.

### 12 3 MVP Test Scenarios

| ID    | Scenario                             | Expected result                                              |
|-------|--------------------------------------|--------------------------------------------------------------|
| T 001 | Register and login                   | Customer receives a valid JWT and can read own profile       |
| T 002 | Duplicate email                      | Second registration returns 409 without another user record  |
| T 003 | Customer calls admin API             | Request returns 403                                          |
| T 004 | Admin creates product and stock      | Product appears in public catalog with correct stock         |
| T 005 | Create order with enough stock       | Order is CONFIRMED and stock decreases exactly once          |
| T 006 | Create order with insufficient stock | Request returns 409 and no item stock changes                |
| T 007 | View another customer's order        | Request returns 404 or 403 according to the final API policy |

| ID    | Scenario                    | Expected result                                          |
|-------|-----------------------------|----------------------------------------------------------|
| T 008 | Cancel confirmed order      | Order is CANCELLED and stock is fully restored           |
| T 009 | Repeat cancellation         | Response remains CANCELLED and stock is not added twice  |
| T 010 | Product Service unavailable | Order creation returns 503 and no confirmed order exists |

## 13 Logging Monitoring and Support

### 13 1 Logging

Each service writes structured, readable logs to standard output. A log entry includes timestamp, level, service name, correlation ID, request path, and a safe summary of the result. Developer logs may be human-readable; later environments can use JSON logging.

### 13 2 Health and Metrics

- Actuator health reports application and database readiness.
- Actuator metrics remain local and protected; they are not public customer APIs.
- The demo checklist verifies all three health endpoints before functional testing.
- A future release may send logs, metrics, and traces to centralized observability tools.

### 13 3 Support Procedure

When an end-to-end request fails, the team records the time, endpoint, correlation ID, request conditions, observed response, and relevant service logs in a GitHub issue. The issue must omit secrets and personal data. The service owner proposes a fix through the normal pull request workflow.

## 14 Eight Week Delivery Plan

Work proceeds in parallel after the Week 1 contracts are agreed. Service assignment must be completed before Week 2. Every week ends with code merged through pull requests and a short integration demonstration.

| Week | Focus                             | Required outcome                                                                                                                                          | Owner          |
|------|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|
| 1    | Foundation and contracts          | Create repository structure, branch protection, issues, service skeletons, Docker Compose draft, data models, endpoint contracts, and shared error format | All developers |
| 2    | Authentication and catalog basics | Assign services; implement registration, login, JWT validation, categories, product creation, and initial migrations                                      | Service owners |

| Week | Focus                               | Required outcome                                                                                                          | Owner                          |
|------|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------|--------------------------------|
| 3    | Profiles and product administration | Implement profile and address APIs, product updates, public catalog, stock adjustment, security tests                     | Service owners                 |
| 4    | Inventory reservation               | Implement atomic reserve and release operations, idempotency, inventory tests, and Order Service client stub              | Product and Order owners       |
| 5    | Order creation                      | Implement order schema, create flow, price snapshots, totals, compensation, and integration tests                         | Order owner with Product owner |
| 6    | Order queries and cancellation      | Implement owned order list and detail, cancellation, stock restoration, repeated-request safety, and full security review | All developers                 |
| 7    | System integration                  | Complete Docker Compose, GitHub Actions, Postman collection, end-to-end tests, logging, correlation IDs, and READMEs      | All developers                 |
| 8    | Stabilization and handoff           | Fix defects, run acceptance checklist, finalize SDD alignment, prepare demo data, tag v1.0.0, and present the system      | All developers                 |

## 14 1 Weekly Team Routine

- Start of week: agree on issues, owners, interfaces, and acceptance criteria.
- During the week: keep pull requests small and request review before work becomes difficult to merge.
- Midweek: integrate current service branches in the Docker Compose environment.
- End of week: demonstrate working behavior, record blockers, and update the next week's issues.

## 15 Future Scale Roadmap

The team should expand only after the previous stage has automated tests and stable contracts. Future components are additions to solve observed needs, not requirements for the first eight weeks.

| Stage   | Additions                                                                                     | Reason to begin                                                      |
|---------|-----------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| Stage 1 | Current three-service REST MVP                                                                | Complete the Week 8 acceptance criteria                              |
| Stage 2 | Notification Service, Kafka, order events, email sandbox, API Gateway                         | The team needs asynchronous notifications and one client entry point |
| Stage 3 | Redis cache, resilience patterns, transactional outbox, centralized logs, metrics, and traces | Catalog reads or service failures reveal measurable bottlenecks      |

| Stage   | Additions                                                                                            | Reason to begin                                                             |
|---------|------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Stage 4 | Container registry, Kubernetes, AWS, managed PostgreSQL, managed Kafka, secrets manager, autoscaling | The system needs shared non-local environments and production operations    |
| Stage 5 | Payment Service with provider sandbox, idempotency, webhooks, refunds, and security review           | The order lifecycle is stable and the team is ready for financial workflows |

## 15 1 Changes Required for Larger Scale

| Concern             | MVP                            | Later design                                                 |
|---------------------|--------------------------------|--------------------------------------------------------------|
| Client routing      | Direct service URLs            | API Gateway and centralized routing                          |
| Authentication keys | Shared HS256 secret            | Asymmetric signing, JWKS, rotation, and secrets manager      |
| Order events        | Synchronous REST only          | Kafka events with outbox and idempotent consumers            |
| Catalog reads       | PostgreSQL queries             | Redis cache after measurement                                |
| Resilience          | Timeouts and controlled errors | Circuit breakers, retries for safe operations, bulkheads     |
| Deployment          | Docker Compose                 | Kubernetes with health probes and autoscaling                |
| Observability       | Local logs and Actuator        | Central logs, dashboards, alerts, and distributed tracing    |
| Database scale      | One local database per service | Managed databases, backups, replicas, and migration controls |

## 16 Risks and Mitigations

| ID    | Risk                                    | Priority | Mitigation                                                                              |
|-------|-----------------------------------------|----------|-----------------------------------------------------------------------------------------|
| R 001 | Scope grows beyond eight weeks          | High     | Keep payment, notification, frontend, Kafka, and cloud work out of the MVP              |
| R 002 | Developers implement incompatible APIs  | High     | Review OpenAPI contracts in Week 1 and integrate every week                             |
| R 003 | Direct changes break main               | High     | Protect main; require pull request review and passing checks                            |
| R 004 | Stock becomes incorrect                 | High     | Use database transactions, version checks, reservation IDs, and duplicate-request tests |
| R 005 | A service fails after stock reservation | Medium   | Use release compensation and log a correlation ID for follow-up                         |

| ID    | Risk                                      | Priority | Mitigation                                                                                        |
|-------|-------------------------------------------|----------|---------------------------------------------------------------------------------------------------|
| R 006 | Secrets enter GitHub                      | High     | Use .env, .gitignore, .env.example, pull request checks, and immediate secret rotation if exposed |
| R 007 | Merge conflicts delay integration         | Medium   | Keep service folders separate, use small pull requests, and update branches frequently            |
| R 008 | Beginners are blocked by unfamiliar tools | Medium   | Prepare service guides, pair on first migrations and security setup, and record setup steps       |
| R 009 | One developer becomes unavailable         | Medium   | Require reviews, shared READMEs, and weekly demonstrations so knowledge is not isolated           |
| R 010 | Demo environment is unreliable            | Medium   | Use Docker Compose health checks, seeded demo data, and a rehearsed Postman collection            |

## 17 Acceptance and Handoff

### 17 1 MVP Acceptance Criteria

- [ ] A new developer can clone the repository and start the complete system from the root README.
- [ ] All three services and all three databases report healthy in Docker Compose.
- [ ] A customer can register, log in, manage an address, browse products, create an order, view it, and cancel it.
- [ ] An administrator can create and update products and adjust stock through secured APIs.
- [ ] Order creation reserves stock atomically and stores item and address snapshots.
- [ ] Order cancellation restores the exact stock once; a repeated request does not duplicate the release.
- [ ] Customers cannot access another customer's profile, addresses, or orders.
- [ ] Public, customer, administrator, and internal endpoints enforce the documented access rules.
- [ ] Each service uses only its own PostgreSQL database and Flyway migrations.
- [ ] Unit, integration, and end-to-end tests required by this document pass.
- [ ] GitHub main is protected; the final code reached main through reviewed pull requests.
- [ ] No payment, notification, frontend, Kafka, Kubernetes, or cloud code is required for MVP approval.

### 17 2 Definition of Done for a Service

- [ ] The service builds with ./mvnw clean verify.
- [ ] The service starts independently with documented configuration.
- [ ] Flyway creates the database schema from an empty database.
- [ ] The service exposes health and OpenAPI endpoints.
- [ ] Business rules, validation, authorization, and errors match this SDD.
- [ ] Automated tests cover normal behavior and important failure cases.
- [ ] The README explains setup, configuration, tests, endpoints, and ownership.

- [ ] A teammate reviewed the pull request and all checks passed.

### 17 3 Design Decisions

| Decision      | MVP choice             | Review trigger                                                   |
|---------------|------------------------|------------------------------------------------------------------|
| Repository    | One monorepo           | Team or release process becomes too large for shared changes     |
| Communication | REST                   | Notification and event workloads require asynchronous processing |
| Data          | Database per service   | Keep as a permanent service-boundary principle                   |
| Security      | JWT with shared secret | Any shared or public deployment                                  |
| Deployment    | Docker Compose         | Need for shared testing or production environments               |
| Payment       | Deferred               | Order lifecycle and security practices are stable                |

### 17 4 Next Documents

After the manager and team approve this SDD, the next deliverables are three beginner-focused implementation guides: User and Authentication Service, Product and Inventory Service, and Order Service. Each guide will include the project structure, dependencies, database migrations, request and response models, class responsibilities, endpoint steps, test cases, Git workflow, and a final run checklist.

## 17 5 Glossary

| Term               | Plain meaning                                                                           |
|--------------------|-----------------------------------------------------------------------------------------|
| API                | A defined way for one program to request data or an action from another program         |
| Microservice       | A small independently runnable application responsible for one business area            |
| JWT                | A signed token that carries user identity and role for protected requests               |
| REST               | HTTP endpoints that exchange requests and responses, usually as JSON                    |
| Database ownership | Only the owning service reads or changes its database                                   |
| Reservation        | A record that temporarily assigns available stock to one order                          |
| Idempotent         | Repeating the same operation has the same result and does not apply the change twice    |
| Compensation       | A follow-up operation that reverses an earlier successful step after a later step fails |
| Migration          | A versioned database change that creates or updates schema objects                      |
| Pull request       | A reviewed proposal to merge a branch into main                                         |
| CI                 | Automated building and testing when code is pushed or a pull request is opened          |